CodeGuard

Inspect AI-generated code before it reaches execution.

AI agents write and trigger operations autonomously. CodeGuard adds an inline, CPU-only security checkpoint before arbitrary runtime execution.

⚠ AGENT OUTPUT SAMPLE UNGUARDED

```
import os os.system("rm -rf /important/data")
```

"Would you execute this without checking?"

Presented by:

Shreyash Bhor

Apex Solutions

SY B.Tech IT VJTI Mumbai

~20 ms
Reported p95 (held-out test)

0 External LLMs
100% local CPU inference

9 Categories
Documented threat taxonomy

Fail-Closed
Blocks when checking fails

THE EXECUTION GAP

Where Does the Security Check Occur?

Traditional Post-Submission Workflow

DELAYED SCAN

Code Submitted → Execution / Build → Later Scan

"Depending on where scanning occurs, harmful behavior may already have been triggered before detection."

CodeGuard Pre-Execution Check

INLINE GATE

Code Generated → CodeGuard Check → Allow or Block

"Checked before the guarded execution step — inspects submitted code before runtime starts."

CORE SOLUTION

5-Stage Hybrid Detection Architecture



Rules + ML → Combined risk assessment

Deterministic fusion · No external LLM call

DEEP INSPECTION

Deobfuscation & Sensitive Data Flow

Obfuscated code can hide dangerous operations.

Selected unwrapping exposes concealed commands for static analysis without executing them.

```
# Before (encoded input): eval(base64.b64decode(encoded)) # Decoded content shown for analysis: __import__("os").system( "rm -rf /example" )
```

Example of a revealed destructive operation. (Analyzed statically, never run).

Follow sensitive data toward its destination.

Source-to-sink tracking flags sensitive origins reaching external egress points.

SOURCE: sensitive credential access

↓ **FLOW:**

SINK: outbound network request

Context and data flow reveal risks that keyword matching alone may miss.

FALSE-POSITIVE CONTROL

Detect Risk Without Blocking Every Lookalike

Unsafe SQL Construction

```
query = ( f"DELETE FROM orders " f"WHERE id = {order_id}" )
cur.execute(query)
```

String interpolation can let untrusted input alter the SQL statement.

VERDICT: BLOCK / FLAG FOR REVIEW

Parameterized SQL

```
query = ( "DELETE FROM orders WHERE id = ?" )
cur.execute(query, (order_id,))
```

Parameterization helps separate SQL structure from user-supplied values.

VERDICT: SAFER PATTERN (NOT A UNIVERSAL SAFETY GUARANTEE)

`ast.literal_eval ≠ eval`

`crontab -l (read) ≠ crontab -r (wipe)`

`capped threads ≠ fork bomb`

`docstring note ≠ real credential`

TAXONOMY

Nine Threat Categories. One Security Checkpoint.

DATA & SECRETS

secret_access
Credentials, keys, shadow files

data_exfiltration
Unauthorized external transfer

ssrf_network_exfil
Internal network / metadata access

SYSTEM INTEGRITY

destructive_command
Destructive file/disk wipes

obfuscated_execution
Concealed commands & decode-exec

persistence_or_privesc
Persistent hooks & sudo escalation

resource_exhaustion
Fork bombs & socket flooding

SUPPLY CHAIN & AI

dependency_risk
Suspicious package typosquats

ai_targeted_instructions
Malicious prompts in docstrings/CI

PROOF OF FUNCTIONALITY

Does the Guard Actually Stop the Execution Step?

demo/intercept_check.py

```
$ python demo/intercept_check.py

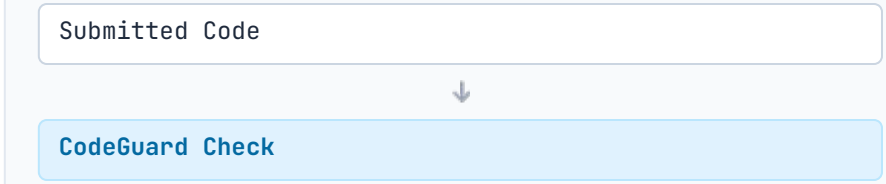
1. BENIGN TEST:
                                [CREATED]

2. MALICIOUS TEST:

                                 [DOES NOT EXIST]

3. ASSERTIONS: PASSED (EXIT CODE 0)
```

DECISION FLOW



ALLOW → Guard permits next step

BLOCK → Execution refused

EVALUATION

Fast Analysis, Evaluated Across Three Test Suites

Dataset	Precision	Recall	Verdict F1	False-Positive Rate	p95 Latency
Dev Sample (42)	1.000	1.000	1.000	0.00%	22.5 ms
Held-Out Test (Family Split)	1.000	0.984	0.992	0.00%	20.6 ms
Adversarial Mutated Suite	0.994	0.983	0.989	1.39%	24.9 ms

MEMORY FOOTPRINT
~180 MB RSS

MODEL ARTIFACT
3.2 MB (Joblib)

ANALYSIS WATCHDOG
2.0s Hard Ceiling

*Reported repository benchmarks; synthetic and mutation-based test results do not establish equivalent performance on independent real-world attacks.

DEPLOYMENT BOUNDARY

Protect AI Agents at the Tool-Execution Boundary

Tool Wrapper Interface

```
@guarded def execute_tool(code: str) → str: # Check runs
before wrapped # operation proceeds return subprocess.run(
code, shell=True )
```

Wraps tools to gate execution without restructuring the host agent framework.

GUARDED EXECUTION LIFECYCLE

1. AI agent generates a code payload.
2. CodeGuard statically checks the payload (~20 ms).
3. **ALLOW:** Application proceeds to next execution step.
4. **BLOCK:** Guard refuses; agent regenerates or requests human review.

*CodeGuard provides pre-execution filtering; it is complementary to, not a replacement for, runtime sandboxing.

LIVE INTERACTION READY

Generated by AI. Checked by CodeGuard. Then Allowed or Blocked.

Fast, local source-code analysis with explainable security findings and a guarded execution interface.

Dashboard: https://codeguard-bdmu.onrender.com/ui/

Repository:

https://github.com/shreyash-2006/CodeGuard

API Check:

https://codeguard-bdmu.onrender.com/check

Ready for a live security challenge.

Choose a snippet. Inspect the verdict. Challenge the guard.